



Skills as Code

*How BootContext makes agent behavior reproducible
across the team and across time.*

*“ The harness asks the agent nothing beyond "do your contract." Everything else is
loaded. ”*

IN ONE PAGE

What it is. Skills are first-class code — Markdown procedures with JSON output schemas, versioned alongside the source they govern. Identity is first-class state — each agent has a profile that owns its long-term memory and operating manual. Between them sits a single primitive: **BootContext**, a harness-assembled JSON envelope that arrives with the agent at first token of stdout and contains every fact the agent needs.

How it works. The dispatcher claims a story, the assembler reads from five sources (profile, repo, schema, database, filesystem) and writes one merged document, the harness pipes it on `stdin` and drops a forensic copy to `/tmp/ops/boot/<storyId>.json`. Three classes of validation error are hard-fail at assembly — the subprocess never spawns when the contract cannot be honored.

What it enables. Behavior is reproducible: same SHA, same skill, same schema, same contract. Every state mutation emits a typed event in the same database transaction as the row update — the audit trail and the truth are one record. Skills can be authored, reviewed, and version-controlled like any other code in the repo.

I. The Architecture

The pipeline runs on a clean separation: skills carry the procedure, profiles carry the identity, and the harness assembles a complete envelope at the moment of dispatch. The agent reads the envelope once and starts work.

Three Layers, One Envelope

An agent at runtime needs three things to begin work: knowledge of who it is, knowledge of what to do, and knowledge of the situation it is acting on. We give each of these a permanent home.

I Identity — persistent, per-profile

Each agent profile lives at `~/.hermes/profiles/<name>/` and ships with `AGENTS.md`, `SOUL.md`, `tools.md`, and `domain-knowledge/`. The profile is the operating manual, the voice, the persistent learning, the connection strings. Verified at boot to exist; digested into `BootContext.identity`.

II Procedure — versioned in the repo

Each skill lives at `skills/<name>/SKILL.md` with a sibling `*.schema.json`. Authored once, reviewed once, shipped with deployments. Every agent of that role consults the same body at the same SHA. The Markdown is the source of truth.

III Situation — computed per-dispatch

Story brief, epic brief, prior outcomes, workspace state, GitNexus impact, doctrine digest, resolved curl endpoints. Built fresh by the assembler. Auditable forever via `/tmp/ops/boot/<storyId>.json`.

Bringing these three together is the responsibility of the **BootContextAssembler**. The dispatcher claims a story, calls `assembler.assemble(input)`, attaches the resulting envelope to the agent's invocation, and only then spawns the subprocess. The agent's first read is the envelope; its first write is on the contract surface the envelope describes.

* THE CONTRACT SURFACE

An agent's contract surface is the JSON output schema named by its skill. The schema is fully resolved (all `$ref`s inlined) before it leaves the assembler. The agent's output is validated against this schema by **ContractGate** at `/complete`. There is one validation rule, applied uniformly, and it is authored alongside the procedure that produces the output.

🍷 II. The Doctrine — Thin Harness, Fat Skills

The harness orchestrates. Skills perform. The user composes.

Our project doctrine — written in `CLAUDE.md` — splits the system into two layers and insists the line is bright.

I

Latent Layer — fat skills

Skills carry process, judgment, quality bars, prose reasoning, domain vocabulary. They live as Markdown files in the repo (`skills/<name>/SKILL.md`) and ship with code. A skill author writes the procedure once, with examples, anti-patterns, failure modes, and a JSON output schema. Every dispatch of that skill consults the same source of truth.

II

Deterministic Layer — thin harness

The harness runs the loop, reads and writes files, manages context, enforces safety, and routes contributions. It does **five things** and nothing more. It does not know what kind of work the skill is doing. It does not have a hardcoded list of roles. It does not encode `if architect then...` branches. The seam is the contract; the contract is the skill's output schema.

The Reproducibility Theorem

Because skills are code, the same git SHA produces the same skill text, the same schema, the same quality bar, and the same procedure. Two agents on the same role at the same SHA — dispatched against the same story — receive an identical contract and consult an identical procedure. The model layer is non-deterministic; the contract surface around it is not.

Same SHA, same skill, same schema, same contract. Three agents with the same role on the same story will produce structurally equivalent output that the same gate validates against the same rules. That is what we mean by reproducibility.

The unit of trust is the contract, not the agent. A reviewer can verify behavior by reading the skill body in the repo, just as a human reviewer reads any other source file. A new operator can onboard by reading skills, not by inheriting tribal knowledge. A change to a procedure flows through the same pull-request discipline as a change to any production code.



III. The BootContext Primitive

*Skills are the contract. Profiles are the identity. **BootContext** is the single envelope that brings them together at the moment of dispatch.*

When the dispatcher claims a story for an agent, the next call — before any subprocess is spawned — is `bootContextAssembler.assemble(input)`. The assembler reads from five separate sources, merges them, and writes one JSON document. That document is delivered to the agent in two ways: piped on `stdin` as part of the invocation envelope, and dropped to `/tmp/ops/boot/<storyId>.json` as a forensic record.

Five Sections, Five Sources

SECTION	CONTAINS	SOURCE
identity	role, agentId, profilePath, soulPrime, doctrineDigest	Profile dir <code>~/.hermes/profiles/<name>/SOUL.md</code> + a 500-token digest of <code>CLAUDE.md</code>
contract	skillName, skillVersion, outputSchema (resolved), finalCurl, fallbackCurl, authToken	<code>skills/<name>/*.schema.json</code> in the repo, with <code>\$ref</code> s inlined
procedure	full SKILL.md body, references, qualityBar, examples	<code>skills/<name>/SKILL.md</code> in the repo (uncapped, full text)
context	epicBrief, storyBrief, priorStageOutputs, priorOutcomeHistory, priorRejection	Database — pipeline_epics, pipeline_stories, outcome_history, skill_outputs
workspace	branch, developTipSha, workingDir, modifiedFiles, srcListing, GitNexus impact	Filesystem + git + GitNexus impact analysis

What's Capped, What's Not

The skill body itself is **not capped**. Whatever length the skill needs to be — three hundred lines or a thousand — the entire body lands in the envelope as raw markdown. The output schema is fully resolved with all `$ref` includes. The quality bar and worked examples ship in full.

Caps apply only to *ancillary references*: when a skill cites an ADR or a flow document, the assembler pulls a 200-token excerpt around the cited section so the agent has the relevant snippet inline without injecting the full reference document. The doctrine digest is similarly capped at 500 tokens. These caps protect the context window from being eaten by ancillary material; they do not constrain the skill itself.

✦ SIZING DISCIPLINE

A typical implementer dispatch produces a BootContext of **seven to fifteen thousand tokens** total — full skill body, full schema, full epic + story brief, prior outputs, workspace snapshot, GitNexus impact. The agent's context window sees this once and reads it once. There is no second pass to find missing pieces.

🍷 IV. Implementation

The substance lives in two files: a port that defines the shape, and an adapter that assembles it.

The Port

`src/ports/boot-context.port.ts` declares `BootContext` and the `BootValidationError` discriminated union. Five interfaces — `Identity`, `Contract`, `Procedure`, `Context`, `Workspace` — compose into one root `BootContext`. The error union enumerates every failure mode by kind:

```
export type BootValidationError =
  | { kind: 'story-not-found';          storyId: string }
  | { kind: 'epic-not-found';          epicId: string }
  | { kind: 'skill-md-missing';         skillName: string }
  | { kind: 'skill-schema-invalid';     skillName: string; detail: string }
  | { kind: 'profile-missing-agents-md'; profilePath: string }
  | { kind: 'profile-missing-tools-md'; profilePath: string } // soft-warn
  | { kind: 'workingdir-missing';       path: string }
  | { kind: 'doctrine-digest-stale';    lastRefresh: string | null }
  | { kind: 'storage-error';            detail: string };
```

Three of those kinds are **hard-fail**: `skill-md-missing`, `skill-schema-invalid`, and `profile-missing-agents-md`. Hitting any of them aborts the boot before the subprocess is spawned. The dispatcher rolls back the claim and emits a `pipeline.boot.failed` event. Two are **soft-warn**: `profile-missing-tools-md` and `doctrine-digest-stale`. The boot proceeds; the agent receives an annotation in `identity` noting the missing piece.

The Adapter

`src/services/boot/boot-context-assembler.ts` is the production `IBootContextAssembler`. Its `assemble()` method walks twelve steps, failing on the first hard-fail kind it encounters:

1. Load story by id; verify role + skillName + acceptance criteria + visibleAt.
2. Load epic by story.epicId; verify recipe linkage and metadata.
3. Verify `workingDir` exists on disk; soft-regenerate via `IGitAdapter.createWorktree` when missing.
4. Verify profile dir exists; verify `AGENTS.md` exists (hard-fail otherwise).
5. Read `SOUL.md` first paragraph; soft-skip when absent.
6. Read `tools.md`; soft-warn when absent.

7. Locate `skills/<skillName>/SKILL.md` (hard-fail otherwise); read body verbatim.
8. Resolve `skills/<skillName>/*.schema.json` with all `$ref` s inlined; ContractGate validates the resolved schema is well-formed.
9. Resolve frontmatter `references[]` : open each cited file, extract a 200-token excerpt around the cited section.
10. Compute the doctrine digest from `CLAUDE.md` 's Spirit + Hard Rules sections (cached; refresh on file change).
11. Query DB for prior stage outputs, outcome history, and prior rejection.
12. Snapshot workspace: branch, develop tip, modified files since develop, src/ listing, GitNexus impact pre-fetch on `storyBrief.targetFiles` .

Each step records timing into `/tmp/ops/boot/<storyId>.assembly-timings.json` . The full BootContext is written to `/tmp/ops/boot/<storyId>.json` with the `authToken` redacted. The version that goes into the stdin envelope retains the token in-memory only.

Wiring

`Dispatcher._dispatchOne()` calls the assembler immediately after the story claim and immediately before `harness.submitTask()` . On success, the BootContext is attached to `task.context._bootContext` . On hard-fail, the dispatcher rolls back via the StoryAggregate path — the same primitive used for spawn-failure rollback — and the row returns to `ready` within milliseconds.

✦ THE REPRODUCIBILITY VERIFIER

Run the assembler twice for the same `storyId` at the same git SHA: the two JSON outputs are byte-equal modulo timestamps. This is the mechanical guarantee behind the reproducibility claim. A test fixture asserts this property in CI.

🍷 V. The Matrix Payoff

In the movie, skills are loaded into the agent at runtime by an external operator. The agent stands up and is ready. Our system operates the same way.

When `ops-implementer-3` is dispatched against a story, the agent's first token of stdout is preceded by a complete envelope:

✦ WHAT IT KNOWS

Its identity (role, profile path, voice). Its contract (skill name, output schema, destination URL, auth token). Its procedure (full SKILL.md, quality bar, examples). The doctrine it must follow. The story brief and epic brief. Every prior `/complete` on this epic, ordered by `completedAt`. The workspace state. GitNexus impact analysis on the target files. The `visibleAt[]` entries it must verify by hand-off.

✦ WHAT IT NEVER HAS TO DO

Grep its profile directory. Locate its skill file. Find the schema. Compute the doctrine digest. Query the database for prior outcomes. Run `git log` to see what changed. Run impact analysis. Resolve `$ref`s. Parse frontmatter. Discover where to POST results.

Behavioral Properties That Hold By Construction

1. **Failures are loud at the right layer.** A missing `AGENTS.md` aborts the boot in milliseconds with a structured error. The dispatcher receives a typed event and rolls back. The dashboard surfaces a red dot. An agent never enters the work loop with a partial map.
2. **Intent is auditable.** Every dispatch is captured in `/tmp/ops/boot/<storyId>.json`. Six months from now, given a story id, an operator can reconstruct exactly what the agent saw at boot — every fact, every digest, every prior outcome. Combined with the outbox event log, the full lineage of who decided what is visible.
3. **Behavior is reproducible across the team.** Two operators run the same epic at the same SHA. The same skill is consulted, the same procedure is followed, the same schema validates the result. This is the foundation on which the workflow factory's promise rests: the same workflow runs the same way for every team that adopts it.

✦ STATE AND EVENTS LIVE TOGETHER

BootContext is the dispatch half of a larger property. The runtime half is the L1–L4 event foundation: every state mutation passes through a domain aggregate that records an event, the row update and the event insert ride one database transaction, a database trigger fires as a safety net for any path that bypasses the aggregate, and external signals (Hermes hooks, ContractGate, dispatcher decisions) flow through the same outbox. The audit trail and the truth are one record. Anomaly detection becomes a one-line SQL query.

Skills as code. Identity as state. Context as envelope. The harness asks the agent nothing beyond "do your contract." Everything else is loaded.

🔧 VI. Standard Operating Procedure

Adding a New Skill

1. Create `skills/<skillName>/SKILL.md` following the canonical 12-section template (Purpose, Contract, Inputs, Procedure, Quality Bar, Output Format, Failure Modes, Self-Review, References, Examples, Anti-Patterns, When Scope Expands).
2. Author the JSON output schema at `skills/<skillName>/<skillName>.schema.json`. Use `$ref` for shared types.
3. Add a `references[]` block in the SKILL.md frontmatter listing every doc the skill cites — path, section, why.
4. Run `npm run verify-profiles` to assert the skill is reachable from every recipe stage that names it.
5. Reference the skill from a recipe stage. The boot-time validator will fail the boot if the schema is malformed.

Debugging a Failed Boot

1. Look at `pipeline_outbox` for the `pipeline.boot.failed` event for the relevant `storyId`. The payload contains the `BootValidationError.kind`.
2. Map the kind to a fix: `skill-md-missing` means a recipe references a skill that is not in `skills/`. `profile-missing-agents-md` means a profile drift; run the sync script. `workingdir-missing` means the worktree was wiped — handled automatically by the regenerator.
3. Open `/tmp/ops/boot/<storyId>.json` if the boot succeeded but the agent produced unexpected output. Diff the BootContext against another successful run of the same skill at the same SHA.

Verifying Reproducibility

1. Pick a story id that has shipped successfully.

2. Run the assembler against the same story id twice via `npx tsx scripts/replay-assemble.ts <storyId>`.
3. Diff the two JSON outputs (modulo the `assembledAt` timestamp). They must be byte-equal.
4. If they are not, file a bug: an upstream input is non-deterministic and must be pinned.

VII. Closing

Skills as code is what fat-skills doctrine looks like when it ships. The procedure that an agent follows is no longer a prompt fragment whispered at runtime; it is a Markdown file in the repo, reviewed by humans, validated by schema, and shipped under version control. Identity is no longer an afterthought; it is a profile directory whose presence is verified at boot and whose contents are digested into the envelope.

BootContext is the seam between the two. It assembles once, it delivers once, it is auditable forever. The harness asks the agent nothing beyond the contract. The agent stands up and the work begins.

Skills are code. Identity is state. Context is envelope. *The harness asks the agent nothing beyond "do your contract."*



Hermes carries the message. The message arrives whole.